IS CYBERNEXT 2026 · CYBERSECURITY TRACK

ReverseShield

Reverse-engineering exposure workbench — complete project report

Document	Value
Version	0.5.0-team
Date	11 November 2026
Status	DRAFT — owner review; not submitted
Prepared by	Naser · Zayed · Nomai · Faisal
Scope	Local static-analysis workbench for Linux ELF and Windows PE
Evidence basis	Recorded Day 4 runs, 11 November 2026

How to read this report

This report explains the prototype and separates measured evidence from claims. Numbers name their source and scope. The project never executes a target, installed application or bundled sample on the host; the only guest execution is a separate deterministic sample-driver check in QEMU.

Document Control

This document is the complete technical explanation of the ReverseShield prototype as it exists in the local Day 4 workspace. It is written for the IS CyberNext evaluation: a technical judge should be able to understand the problem, the design, the measured evidence, and the limits without reading the source first.

Item	Value
Document	Full project report (DRAFT)
Audience	IS CyberNext judges, technical reviewers, team handover
Language	English; Arabic explanation available verbally
Software version	0.5.0-team, branch feature/ollvm-protection
Primary platform	Fedora Workstation 43, Python 3.14, loopback UI
Secondary platform	Windows 10/11 workbench subset; manual acceptance pending
What this is	A static-analysis and evidence workbench, demonstrated on self-authored samples
What this is not	A production protector, antivirus, CVE scanner, or dynamic malware sandbox

Both this report and the accompanying slide deck are marked DRAFT. They are generated locally for owner review and are not the submission copy. GitHub publication, release tagging and IS CyberNext registration remain separate owner decisions.

How to read the evidence

Numbers in this report are either (a) produced by the project test suite, (b) recorded in docs/evidence/*.json with timestamps and hashes, or (c) measured by a live workbench run on 11 November 2026. Where a number is a count of tool output rather than a security property, the report says so. N/A means unmeasured, never zero.

Contents

Page numbers refer to this PDF build. The Word file uses the same section order.

Section	Page
1. Executive Summary	04
2. Problem, Users and Scope	05
3. Threat Model and Safety Boundary	05
4. Solution Overview	06
5. Architecture	07
6. Analysis Engine and Evidence Model	08
7. Protection Profiles and Transformations	09
8. Behaviour Verification in QEMU	10
9. Interface and Operator Experience	11
10. Security Engineering of the Workbench	12
11. Evaluation and Measured Evidence	13
12. Related Work and Differentiation	14
13. Limitations and Non-Claims	15
14. Roadmap and Release Gates	16
15. Team, Workflow and Handover	16
16. Conclusion	17
Appendix A. Test Inventory	17
Appendix B. Evidence Register	18
Appendix C. Reproduction Commands	18
Appendix D. Security Scan Details	19
Appendix E. Acceptance Matrix Summary	19
Appendix F. Glossary	20

1. Executive Summary

ReverseShield is a local, browser-based workbench that shows what a compiled program reveals to a reverse engineer, then lets an operator change a **copy** and measure what the change actually removed. It analyzes Linux ELF and Windows PE files with established static tools, presents the result in plain language, and compares before/after reports using qualified metrics rather than a single score.

The project exists because two questions are surprisingly hard to answer together: “What does our build expose?” and “Did the change we made actually reduce that exposure?” Commercial protectors usually answer with a claim. Vulnerability scanners answer a different question. Manual reverse engineering can answer both, but it is expert-only and hard to reproduce. ReverseShield turns that work into a repeatable local workflow with visible evidence and explicit limits.

What was built

- A Python standard-library server and a dark, terminal-inspired web interface served on loopback only.
- Static analysis adapters for readelf, objdump, strings, nm and optional project-local Ghidra headless.
- Discovery for uploaded files, folders, Flatpak applications and RPM payloads, plus six bundled self-authored samples.
- Four clearly bounded transformation profiles: symbol removal, demo constant blinding, binary metadata hygiene, and OLLVM source obfuscation for the five original samples.
- Exploit-mitigation checks for ELF and PE, reported as observed properties rather than a score.
- A QEMU micro-VM harness that checks separate deterministic driver builds without disk or network.
- A Day 4 evidence chain: acceptance matrix, claim map, security review, measured hardening results, package checksums and a silent demo.

Measured results at a glance

Result	Value	Where it is recorded
Automated tests	167 tests, 1 skipped, passed	docs/evidence/verification.json
Test suite with Ghidra enabled	Run on Fedora, 0 failures, 0 errors	verification.json (RUN_GHIDRA_TESTS=1)
Extracted Lite ZIP	167 tests, 7 skipped, passed	package extraction check, Day 4
Deterministic QEMU vectors	5 of 5 matched	docs/evidence/ollvm-five-qemu.json
Hardened fixture	NX, PIE, full RELRO, canary and FORTIFY present	docs/evidence/hardening-level5.json
Bandit	0 high · 3 medium · 113 low	docs/evidence/security-day4.json
Lite package	286,962 bytes	dist/

What the project does not claim

ReverseShield does not claim that any binary is secure, unhackable or free of vulnerabilities. It does not detect CVEs, score exploitability or prove behavioural equivalence. Its OLLVM profile applies only to the five original self-authored sample sources; arbitrary third-party binaries are analysis-only. Native Windows manual acceptance has not been performed. These boundaries are repeated in the report because they are part of the engineering, not a disclaimer added at the end.

One-sentence summary

ReverseShield is an evidence-first static workbench: it makes binary exposure visible, measures the effect of a bounded change, and states clearly what the evidence cannot prove.

2. Problem, Users and Scope

A compiled program is not opaque. Even without source code, a binary carries names, paths, messages, library imports, function boundaries and sometimes full debug information. A reverse engineer uses those artefacts to reconstruct behaviour. The same artefacts are often invisible to the team that shipped the build, because release engineering, security review and product development look at different outputs.

The practical problem

Consider a small policy engine. The original Level 5 sample used in this project ships with 35 ELF symbol-table entries, debug information, compiler notes, source-folder names and six plaintext policy messages. `strings` alone reveals the message text; `readelf` shows the symbol table and debug sections; Ghidra recovers the policy function and its decision logic. None of this is a software vulnerability in the CVE sense, but all of it lowers the cost of understanding the program.

Teams usually respond in one of three ways. They buy a protector and trust a claim; they run a vulnerability scanner and receive a different kind of answer; or they ask an expert to look at the binary once. None of these produces a repeatable before/after record that a non-specialist can inspect. ReverseShield is aimed at that gap.

Who the workbench is for

- Student and small product teams that need to see their own build without hiring a reverse-engineering specialist.
- Maintainers who want a defensible record of what changed after a stripping or obfuscation step.
- Security reviewers who need a local, auditable workflow that does not upload proprietary binaries.
- Judges and reviewers who want claims tied to artefacts rather than adjectives.

Scope of this version

The current release is deliberately narrow. It supports ELF and PE static analysis, a set of local transformations on copies, and a bounded QEMU check for self-authored sample drivers. It does not support live process analysis, kernel modules, firmware, managed code, mobile packages, or automatic exploitation. Source transformations are restricted to the bundled sample corpus. The sixth sample, `level-5-hardened`, is selectable for analysis but excluded from source transformation because it exists to demonstrate mitigation properties, not to be rebuilt.

In scope	Out of scope
ELF and PE static inspection	Executing external targets on the host
Symbols, strings, sections, imports, functions, call graph	CVE/CWE detection or exploit generation
Copy-only stripping, constant blinding, metadata hygiene	Protecting arbitrary third-party binaries
OLLVM source passes for five self-authored samples	Full behavioural equivalence or fuzzing
Hardening property observations	A security score or certificate
Separate deterministic driver checks in QEMU	Dynamic analysis of installed applications

3. Threat Model and Safety Boundary

The safety model is the most important design decision in the project. ReverseShield analyzes untrusted bytes with static parsers, never loads a target into the host process, and never executes a target, an installed application or a downloaded sample. The only execution path is a separate QEMU guest running deterministic drivers compiled from the project's own sample sources.

Assets and adversary

The asset under study is the information content of a build: names, debug paths, strings, imports, structure and readable logic. The modelled adversary is a reverse engineer with the binary and commonly available tools. The project does not model a remote attacker, a memory-corruption exploit, a supply-chain compromise or a privileged local attacker. Those are different problems and would need different evidence.

Boundary	Rule enforced in this version
Host execution	No target, installed app, EasyEffects process or sample is executed on the host.
Modification	Original files are copied into a job directory; transformations write new files.
Third-party input	Installed applications and RPM payloads are discovered and read, not run or modified.
Source transforms	Only the five original self-authored sample sources; the sixth hardened fixture is analysis-only.
Guest execution	Separate deterministic drivers, no disk, no network, serial output only.
Network	The server binds 127.0.0.1; no upload to a cloud service and no remote analysis API.
Authorization	Jobs require an explicit confirmation that the input is benign and the operator is authorized.
Publication	Private paths, raw reports, keys and credentials are excluded from packages and evidence.

Why this is not a sandbox

A static parser still processes hostile input. The project uses bounded subprocess calls, timeouts, output caps, argv-only invocation and no shell interpolation, but it does not claim process isolation for readelf, Ghidra or the Python parsers. A production system would add containerisation, seccomp and resource limits. For the event prototype, the honest statement is: static parsing of untrusted bytes, with bounded commands, inside a single-user loopback service.

> The confirmation checkbox records operator intent; it does not prove ownership. The project is intended for benign, authorized analysis only. This limitation is stated in the interface and in the acceptance matrix.

4. Solution Overview

The workflow is a five-step loop: select an input, analyze it, transform a copy, reanalyze the copy with the same tools, and compare the two reports. The loop is visible in the interface and reproducible from the job record. Nothing is hidden behind a single “protect” button, because the interesting question is not whether a button ran but what changed.

```
Select input  ->  Analyze original  ->  Transform a copy
   ^                                          |
   |                                          v
 Evidence  <-  Compare qualified metrics  <-  Reanalyze copy
```

Two report layers

The first layer is **exposure evidence**: what the selected tools recovered from the file. It includes names, debug information, strings grouped by kind, imports/exports, sections, functions, decompiled functions, call relationships and graph data. The second layer is **mitigation posture**: observed hardening properties such as NX, PIE, RELRO, stack canary and FORTIFY on ELF, and ASLR/DEP/CFG/cookie flags on PE. The interface keeps these layers visually separate so a mitigation flag is never mistaken for a vulnerability verdict.

Qualified comparison

Every comparison metric names the tool that produced it, because a count without a source is easy to misread. For example, “names reported by selected tools” includes Ghidra labels, while “ELF/PE symbol-table entries” is a direct table count. Removing the symbol table can reduce the second to zero while Ghidra still recovers the function and generates new labels, so aggregate counts may rise. The report explains this instead of presenting a single percentage improvement.

Principle	Implementation
Original untouched	Each job copies the selected target into its own directory.
Same tools on both sides	The copy is reanalyzed with the same selected tools and options.
Measured, not modelled	Metrics are computed from tool output and the actual bytes.
N/A is not zero	Unmeasured fields display N/A with an explanation.
Hashes anchor evidence	Original, copy, driver and transcript hashes are recorded.
Limits stay visible	The report repeats scope and non-claims next to the numbers.

5. Architecture

The system is intentionally small: one Python process, one loopback HTTP server, a static front end, a job engine, tool adapters and a per-job evidence directory. There is no database, no message broker, no cloud dependency and no external service. That keeps the trust boundary inspectable and the package runnable on a laptop.

```
Browser (static HTML/CSS/JS)
  |  token-protected loopback HTTP
  v
Python server (app.py)
  |-- discovery / sample catalog
  |-- workflow job engine  -> runs/<job>/ events + report.json
  |-- analysis adapters    -> readelf, objdump, strings, nm, Ghidra
  |-- protection adapters  -> strip, constant blinding, hygiene, OLLVM
  `-- behaviour adapter    -> QEMU micro-VM (sample drivers only)
```

Component	Responsibility
app.py / HTTP layer	Serves the UI, enforces loopback binding, token, origin and CSP checks, exposes the JSON API.
workflow.py	Creates jobs, validates operation/profile pairs, runs the worker, emits events, stores the result.
analysis.py / engine.py	Invokes static tools with bounded argv commands and normalises their output.
metrics.py	Computes counts, entropy, symbol/string categories and comparison deltas.
hardening.py	Reads ELF/PE headers and imports to report mitigation properties.

Component	Responsibility
demo_protection.py / binary_protection.py / ollvm_protection.py	Builds transformed copies and records hashes and scope.
behavior_qemu.py	Builds a minimal initramfs, boots QEMU without disk/network, compares deterministic drivers.
sanitize_report.py	Produces a report copy with private paths and sensitive fields removed.

Job lifecycle

A job is created only after target selection and authorization. It receives a private directory under runs/<job_id>, a copy of each target, a queued state and a monotonically increasing event log. The worker moves through running to completed, failed or cancelled. The UI polls the job endpoint; the final result is written as JSON and rendered into the report. Cancellation is cooperative and checked between tool invocations.

Platform support

Fedora Linux is the primary and fully exercised platform. The browser workbench, static analysis, ELF/PE reports, hardening checks and packaging run there. On native Windows the workbench subset is implemented but not manually accepted; the OLLVM toolchain and the QEMU workflow are not: the interface and backend report them as unavailable instead of failing late. Windows manual acceptance is an open release gate, not a completed claim.

6. Analysis Engine and Evidence Model

The analysis engine is deliberately conservative. It calls established command-line tools, captures their output, and turns the output into both raw artefacts and normalised signals. It does not attempt to replace Ghidra or to infer intent. The value is the consistent evidence model around the tools.

Tool adapters

Tool	Used for	Notes
readelf	ELF header, sections, symbols, dynamic tags, notes	Primary source for hardening and symbol counts.
objdump	Disassembly and section contents	Bounded output; used for instruction-level context.
strings	Byte-scan strings and categories	Reports URLs, paths, messages and compiler notes separately.
nm	Symbol table inspection	Complements readelf for name recovery.
Ghidra headless	Functions, decompiled C-like code, call graph, labels	Optional, project-local, enabled in the recorded full suite.

Evidence model

Each result keeps three levels of detail. Raw output is preserved for audit. Normalised fields drive the report and comparison. Human-readable findings explain why a signal matters, with references back to the raw evidence. This separation is what allows the same job to produce a judge-friendly summary, a technical table and a machine-readable record without duplicating analysis.

Signal	What is measured	What it does not prove
Names / symbols	Symbol-table entries, recovered labels	That removing names removes logic

Signal	What is measured	What it does not prove
Debug information	Section presence, paths, compiler notes	That the build is vulnerable
Strings	Byte-scan text, categorised by path/message/URL	That a message is a secret
Imports / exports	Dynamic symbols and libraries	Exploitability of an imported function
Functions / pseudocode	Recovered routines and C-like bodies	That the recovery is complete
Call graph / CFG	Tool-reported relationships	Runtime behaviour
Hardening	NX, PIE, RELRO, canary, FORTIFY, PE flags	That the program is secure

Hardening checks

For ELF, the checker reads PT_GNU_STACK for NX, ET_DYN with DF_1_PIE for PIE, PT_GNU_RELRO plus BIND_NOW for RELRO, canary symbols such as __stack_chk_fail, and FORTIFY symbols such as __printf_chk. For PE, it reads DllCharacteristics, load-config fields and cookie symbols. Results are Present, Absent, Partial or Unknown, each with the evidence string that produced it. Presence is not a guarantee; absence is not proof of exploitability.

Design rule: no composite score

The project deliberately refuses to collapse exposure and mitigation into one number. A score would hide which tool produced which signal and would invite claims the evidence cannot support.

7. Protection Profiles and Transformations

ReverseShield offers four transformation profiles. Each is a demonstration of a technique, not a production protection product. Each writes a new copy, records the exact command or compiler invocation, and compares the copy with the original using the same analysis tools.

Profile	Operation	Inputs	What it demonstrates
Symbol removal	`strip_compare`	ELF/PE copy	Removes symbol/debug tables; shows what remains recoverable.
Constant blinding	`protect_compare`	Five original sample sources	Replaces integer literals with XOR expressions and adds an opaque guard.
Binary hygiene	`binary_compare`	ELF/PE copy	Normalises selected build metadata without source.
OLLVM source	`ollvm_compare`	Five original sample sources	Flattening, bogus control flow, substitution, string encryption, anti-decompiler pass.

Symbol removal, measured

On the Level 5 sample, symbol removal reduced ELF symbol-table entries from 35 to 0 and byte-scan strings from 134 to 60. File size fell from 15,904 to 10,784 bytes. The debug sections and symbol table changed from present to absent. Ghidra still recovered the policy function: decompiled functions moved from 10 to 8 and generated labels from 100 to 67, but the logic remained readable. That is the central lesson of the comparison: removing names raises cost modestly; it does not remove logic.

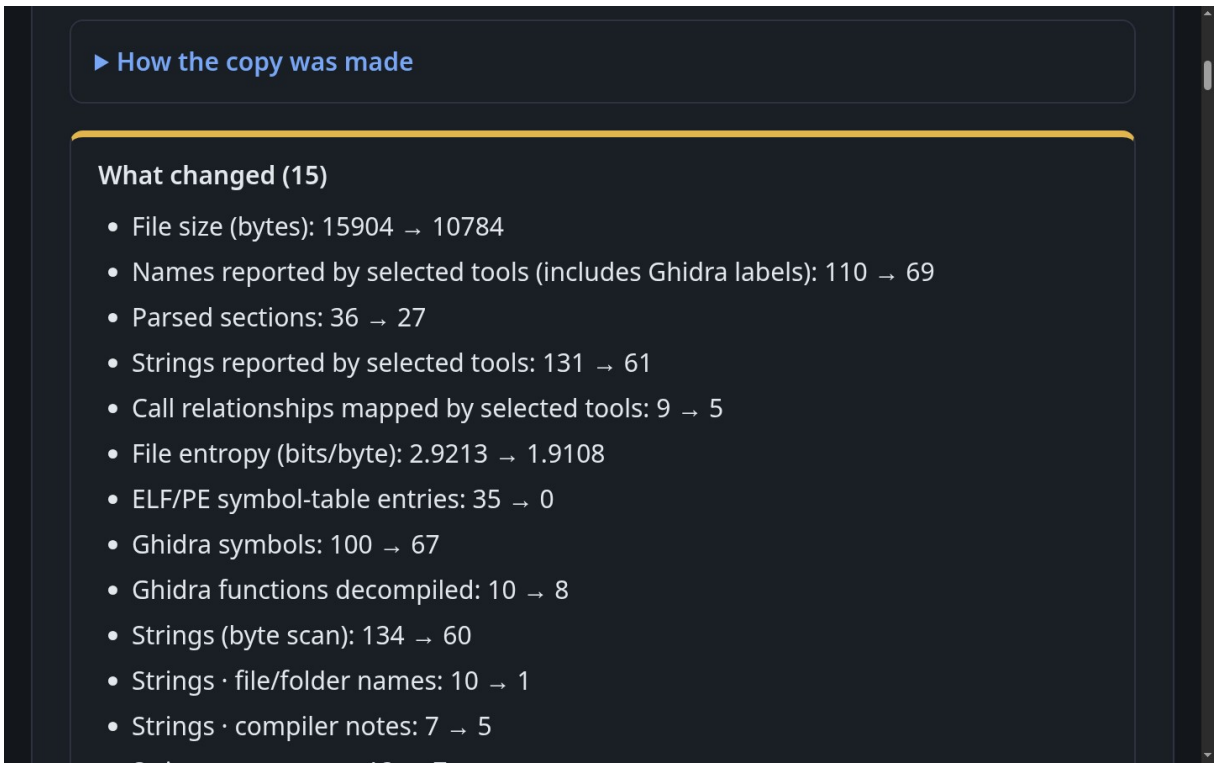


Figure 1. A real symbol-removal comparison in the local workbench, 11 November 2026. Counts name their tool; N/A means unmeasured.

OLLVM source transformation, measured

The OLLVM profile compiles the five original sample sources with a project-local xollvm build on LLVM 22. It applies control-flow flattening, bogus control flow, instruction substitution, string encryption and an anti-decompiler pass, then strips the output. On Level 5, the protected copy grew from 15,904 to 109,296 bytes. All six policy messages disappeared from plaintext, the ELF symbol table fell to zero, and file entropy rose from 2.9213 to 6.4849 bits/byte. Ghidra labels rose from 100 to 594 and analysis took substantially longer. The copy is harder to read casually, not impossible to analyze.

OLLVM and constant blinding operate on the five original self-authored sample sources. A compiler pass cannot transform an arbitrary ELF or PE without source. The sixth sample, level-5-hardened, is analysis-only for this reason.

Binary hygiene

Binary hygiene changes only a copy of an authorized ELF or PE. It removes selected build metadata and records the resulting hashes. It is not a packer and does not claim to defeat a determined analyst. Its purpose is to show that metadata-level changes can be made and measured safely, without executing the artifact.

8. Behaviour Verification in QEMU

The QEMU check exists because a static report cannot show that a transformed program still produces the expected output for a deterministic input. It is intentionally narrow: the project compiles small driver programs from its own sample sources, boots a minimal Linux guest with no disk and no network, runs the original and protected drivers, and compares their serial transcripts and exit codes.

Stage	Implementation detail
Inputs	Self-authored sample source; original and protected driver builds.

Stage	Implementation detail
Packaging	Static busybox/initramfs assembled with argv-only commands and in-process gzip.
Guest	QEMU system emulation, host-provided kernel, no disk image, no network device.
Comparison	Serial transcript SHA-256 and exit code for each side; QEMU return code must be zero.
Evidence	Per-sample hashes, duration, matched flag and explicit scope field.
Failure	A non-zero QEMU return code or a transcript mismatch fails the check.

Sample	Matched	Exit original / protected	QEMU rc
level-1	yes	0 / 0	0
level-2	yes	0 / 0	0
level-3	yes	0 / 0	0
level-4	yes	0 / 0	0
level-5	yes	0 / 0	0

Recorded scope: `separate_deterministic_drivers`. Artifact behaviour verified: `false`. The guest ran separate deterministic driver builds, not the downloadable protected ELF. Matching five vectors is evidence of regression safety for those vectors; it is not evidence of general behavioural equivalence.

> It would be easy, and wrong, to say “the protected binary was tested and behaves the same”. The downloaded artifact was never executed. The UI and evidence records keep `behavior_verified` false for that artifact and report driver-vector matching separately.

Harness hardening

The harness resolves guest library dependencies statically with `readelf` instead of `ldd`, so it never loads a target on the host. It invokes `cpio` with an argument vector and compresses in Python rather than interpolating a shell pipeline. It requires the QEMU return code to be zero, records transcript and driver hashes, and omits raw commands and host paths from the evidence JSON. Spaces and shell metacharacters in output paths are covered by regression tests.

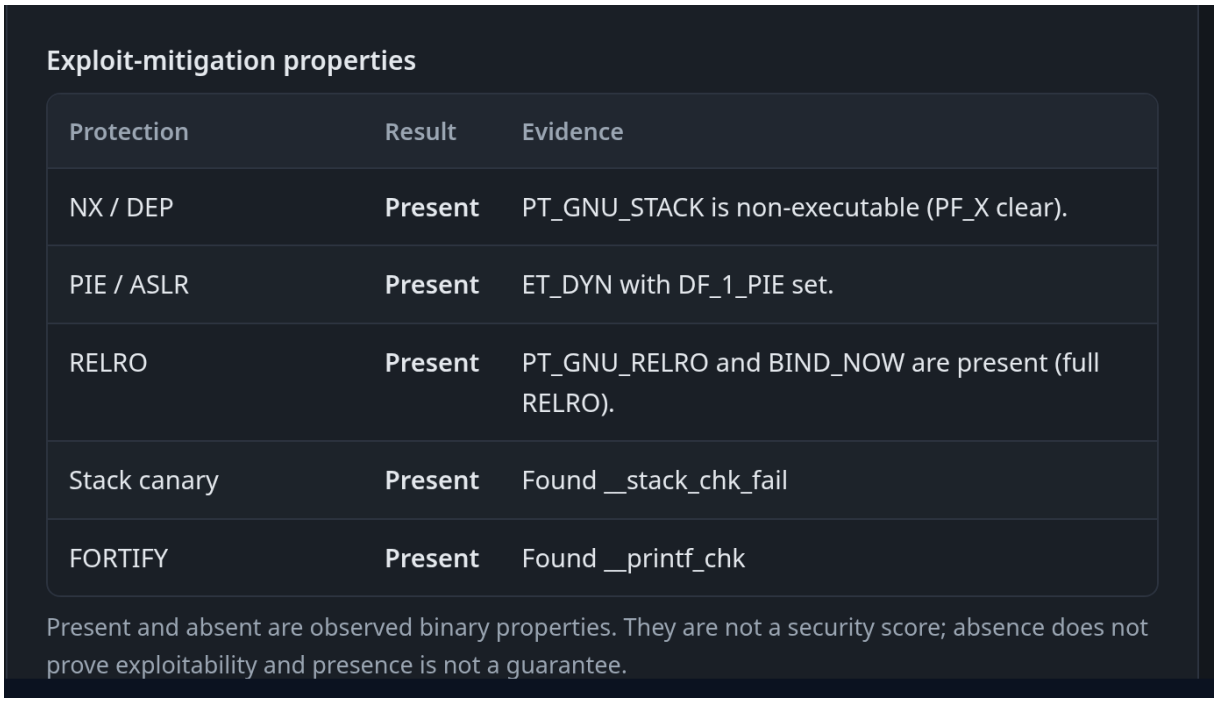
9. Interface and Operator Experience

The interface is a single-page application served from the project. It is deliberately plain: static HTML, CSS and JavaScript, no framework, no build step, no external CDN. The visual language is a dark terminal-inspired palette with a cyan accent, matching the subject matter without pretending to be a commercial SOC console.

Four views

- **Operate** — choose a file, discover targets, authorize, select tools, run analysis or a transformation, read the report.
- **Compare** — original versus changed copy, with “what changed”, “what stayed the same”, scope and download actions.
- **Monitor** — live job events, tool starts/ends, timings, warnings and failure messages.
- **Read Me** — a short operator guide, sample catalog and limitations.

The report uses collapsible sections for raw evidence, so a judge can read the plain-language findings first and open the technical detail only when needed. Screenshots used in the deck and this report come from the live local UI and were checked for private paths.



Exploit-mitigation properties

Protection	Result	Evidence
NX / DEP	**Present**	PT_GNU_STACK is non-executable (PF_X clear).
PIE / ASLR	**Present**	ET_DYN with DF_1_PIE set.
RELRO	**Present**	PT_GNU_RELRO and BIND_NOW are present (full RELRO).
Stack canary	**Present**	Found __stack_chk_fail
FORTIFY	**Present**	Found __printf_chk

Present and absent are observed binary properties. They are not a security score; absence does not prove exploitability and presence is not a guarantee.

Figure 2. Exploit-mitigation properties for the hardened Level 5 fixture, with the evidence string for each observed property.

Accessibility and language

The interface is English-only and has not received a formal accessibility audit. The terminal aesthetic uses high-contrast text, but keyboard-only operation and screen-reader behaviour should be tested before a public release. Arabic explanation is available verbally; the documents are intentionally English for the event.

10. Security Engineering of the Workbench

Securing the workbench is a separate problem from analysing a binary. A tool that reads hostile files must not become the vulnerability. This section describes the controls that exist, the evidence behind them, and the controls that do not exist yet.

Control	Implementation	Evidence
Loopback binding	Server listens on 127.0.0.1 only	app.py, API tests
Session token	Random token required on API requests	API tests, UI contract
Origin / referer / fetch-site	Rejects cross-site requests	test_security_day2/day3
CSP and headers	Strict content-security policy; no inline scripts	UI contract tests
No host execution	Static tools read bytes; copies only	Design + review
Operation/profile pairing	Analysis cannot dispatch a transformation	test_operation_profile, live HTTP 400
Path handling	Job directories under runs/; upload names sanitised	Discovery/API tests
Sanitised export	Private paths and sensitive fields removed	test_security_day3
Package allowlist	Git-index files only; excludes tools/, runs/, raw scans	test_package_selection
Evidence allowlist	Eleven named files; private-path regex guard	test_day4_evidence_pack

A real bug found by review

The Day 4 review found that an `analyze` job could be created with a transformation profile attached. Because the worker dispatched on the normalized profile, a request that looked like analysis could carry a profile into execution. The fix validates the exact operation/profile pair before any job is created. A live request combining `analyze` with a transformation profile now returns HTTP 400, and the regression test asserts the rejection before job creation.

Privacy of the sample corpus

The first sample builds embedded the developer home path in DWARF data. A read-only audit found the leak; the build now uses compiler prefix maps to emit stable synthetic project-relative paths, and all six samples were rebuilt without execution. A regression test fails if a private home path appears in the sample bytes.

Residual risk

- The token is CSRF protection, not authentication or multi-user authorization.
- Static parsers are not a sandbox; a malicious file is still parsed by complex native tools.
- Ghidra is optional and runs with the user's privileges when enabled.
- The sanitizer is pattern-based and cannot inspect compressed PDF text or arbitrary binary encodings.
- No independent penetration test has been performed on the workbench.

11. Evaluation and Measured Evidence

This section separates what was measured from what was merely built. Every table names the source of the number and the scope in which it is valid.

Automated tests

The recorded full suite ran 167 tests with 1 skipped on Linux 7.2.13-100.fc43.x86_64 x86_64 with Ghidra enabled. Result: passed, 0 failures, 0 errors. The extracted Lite package, which does not bundle Ghidra or the OLLVM toolchain, ran 167 tests with 7 optional-dependency skips and no failures. The suite covers analysis, discovery, API security, PE support, RPM handling, hardening, OLLVM scope, QEMU harness safety, packaging, evidence packs, documents and the silent demo.

Hardening evidence

Property	Level 5 original	Level 5 hardened
NX / DEP	Present	Present
PIE / ASLR	Absent	Present
RELRO	Partial	Present
Stack canary	Absent	Present
FORTIFY	Absent	Present

Both files were built non-executable (0600 in the build workspace) and were never executed on the host; a fresh Git checkout may normalise the mode, so the enforced safety property is non-executable. Original: 15,904 bytes, SHA-256 `7963c8cc2fdf8e09…`. Hardened: 12,792 bytes, SHA-256 `2ac29fa1e499fa27…`. Full hashes are in Appendix B.

Comparison evidence

Metric	Original	After symbol removal	Source
ELF symbol-table entries	35	0	readelf
Byte-scan strings	134	60	strings
File size	15,904 B	10,784 B	filesystem
Debug sections	present	absent	readelf
Symbol table	present	absent	readelf
Ghidra labels	100	67	Ghidra
Decompiled functions	10	8	Ghidra
Entropy	2.9213	1.9108	metrics

Behaviour evidence

Five self-authored samples were checked in QEMU with separate original and protected driver builds. All five matched; every original and protected exit code was zero; every QEMU return code was zero. Scope is separate_deterministic_drivers and the downloadable artifact is not behaviour-verified. The sixth hardened fixture is outside this row set.

Security scanning

Bandit 1.9.4 scanned 8,859 lines and reported 116 issues: 0 high, 3 medium and 113 low. The three medium findings are in test code. detect-secrets reported checksums in evidence/setup files and deliberate sanitizer fixtures; no live credential was found. Semgrep and ZAP were not re-run on this tree, so no freshness claim is made for them.

Packaging evidence

Artifact	Size	SHA-256 prefix
Lite Windows/source ZIP	286,962 B	614cfa27f43829b0…
Evidence ZIP	145,593 B	028375e48fc2559d…

The Lite package contains 101 entries selected from the Git index only. It excludes tools/, runs/, raw security scans, demo frames, private handover notes and generated documents. Extraction and a full test run were performed on the extracted copy. The Evidence ZIP contains 11 allowlisted files and is not runnable; it is a review package.

12. Related Work and Differentiation

ReverseShield sits between three established categories. It is not a better packer, not a vulnerability scanner, and not a replacement for an expert with Ghidra. Its contribution is the evidence loop around existing tools: a safe local workflow, qualified metrics, and explicit scope.

Category	Typical output	What ReverseShield adds
Packers / obfuscators	A claimed protection level	Measured before/after evidence and visible residual logic
Vulnerability scanners	CVE/CWE findings and severity	Binary exposure signals and mitigation posture, with no CVE claim

Category	Typical output	What ReverseShield adds
Manual reverse engineering	Expert notes and screenshots	Repeatable jobs, hashes, reports and comparison
Hardening checkers	A flag table	Context: the same toolchain also shows names, strings and logic
Sandboxes	Runtime behaviour	Static-first safety; a separate, narrowly scoped QEMU check for sample drivers

The closest technical neighbours are Ghidra for recovery, binutils for metadata, checksec-style scripts for mitigation flags, OLLVM for source obfuscation, and QEMU for controlled execution. ReverseShield integrates them behind one local workflow and, importantly, keeps their different meanings separate.

What would count as validation

A credible next step is a small benchmark with independent binaries and a blinded before/after review: can a reviewer identify what changed, and does the report change a release decision? The current evidence demonstrates that the workflow runs and measures consistently; it does not yet demonstrate decision impact with external users.

13. Limitations and Non-Claims

The project's credibility depends on saying what it cannot do. The following statements are non-claims, not modest phrasing: they are enforced by scope or left as open gates.

Not claimed	Why / current status
The binary is secure or unhackable	No static property or five-vector check can establish this.
CVE/CWE detection or exploitability scoring	Not implemented; a vulnerability scanner is a different tool.
Full behavioural equivalence	QEMU checks separate drivers, not the downloadable artifact or general inputs.
Protection of arbitrary third-party binaries	OLLVM and constant blinding require the five original sample sources.
Native-Windows full support	Workbench subset only; manual acceptance not performed.
Production sandboxing	Static parsers run in the server process boundary, not a hardened sandbox.
Multi-user or remote service	Loopback, single-user design with a CSRF token, not IAM.
Fresh Semgrep/ZAP results	Bandit and detect-secrets were re-run; Semgrep/ZAP were not.

Open release gates

- Clean Windows 10/11 manual acceptance of the Lite package, including setup.bat, run.bat, PE analysis and report download.
- Independent review of the final PDFs and claim map before external release.
- A decision on GitHub publication versus private ZIP delivery; the current branch is not pushed or merged.
- IS CyberNext account, eligibility, declaration and submission checks by the owner.
- A stable tag and release notes after submission if the owner chooses to publish.

Honesty as a feature

Every limitation in this section maps to an acceptance-matrix row or a documented test boundary. A judge can ask "how do you know?" and receive a file, a hash or a test name.

14. Roadmap and Release Gates

The near-term goal is trust, not feature count. A release is ready when a new operator can install it, run the six samples, read the report, and reproduce the evidence on a clean machine — and when the team can state the limits without hedging.

Gate	Requirement	Status
G1	Fedora workbench runs from the packaged source	Passed locally
G2	Full automated suite green with Ghidra enabled	167 tests, 1 skipped, passed
G3	Evidence regenerated after the final sample rebuild	Passed
G4	Lite and Evidence ZIPs reproducible with checksums	Passed locally
G5	Clean native-Windows manual acceptance	Open
G6	Owner approval, event eligibility and submission	Open — owner action

Next milestones

- **Release readiness:** Windows acceptance, signed/reproducible package, CI on every change.
- **Pilot:** two small teams use the report in a real release review; capture whether it changes a decision.
- **Research:** isolated dynamic lab, SBOM/CVE correlation, more formats, larger benchmark.
- **Product:** accessibility, multi-user separation, and a proper parser sandbox before any hosted use.

15. Team, Workflow and Handover

The project was built as a four-day relay by four students, with a different platform and focus each day. The handover is part of the engineering: each day had to leave a runnable tree, tests and written context for the next person.

Member	Day / focus	Contribution
Naser	Day 1 · Windows and PE	Windows setup, PE acceptance path, first-run reliability.
Zayed	Day 2 · Analysis accuracy	Metrics, Ghidra integration, one-click demo.
Nomai	Day 3 · Security and API	Token/origin controls, sanitizer, SAST/DAST evidence.
Faisal	Day 4 · Integration and release	OLLVM/QEMU scope, hardening evidence, packaging, documents, demo.

The repository uses day branches, pull requests and owner merges. The final Day 4 work is on feature/ollvm-protection and remains local in this workspace. Teammates use Windows, so the package includes setup.bat and run.bat, and the report distinguishes Windows support from Windows manual acceptance.

Handover artefacts

- START_HERE.md, README.md, LITE_PACKAGE.md — orientation and packaging.
- docs/handoff/DAY4.md — state, gates, decisions and next steps.
- docs/evidence/ACCEPTANCE_MATRIX_DAY4.md — platform and release columns.

- docs/evidence/CLAIM_EVIDENCE_DAY4.md — claim-to-evidence mapping.
- docs/demo/ — silent draft video, narration script, storyboard and screenshots.

16. Conclusion

ReverseShield demonstrates a practical middle path between “trust the protector” and “reverse engineer it manually”. It makes binary exposure visible with established tools, changes only copies, measures the result with qualified evidence, and states the limits in the same report as the numbers.

The prototype runs locally, has a green automated suite, produces a real before/after comparison, measures mitigation properties, and includes a narrow QEMU check that is carefully scoped. It is not a finished security product, and it does not claim to be. The remaining work is mostly trust work: Windows acceptance, independent review, pilot feedback, and the owner’s publication and event decisions.

Closing statement

Evidence, not verdicts. ReverseShield shows what a build reveals, what a change removed, and what the team still does not know.

Appendix A. Test Inventory

Test counts were collected with the standard library loader across tests/test_*.py in the current tree. The total matches the recorded suite result.

Test module	Tests
test_analysis	3
test_app_api	8
test_behavior_qemu	9
test_binary_protection	2
test_binary_protection_pe	1
test_day4_documents	8
test_day4_evidence_pack	3
test_day4_test_portability	9
test_demo_protection	3
test_demo_video	4
test_discovery	12
test_engine	7
test_hardening	7
test_metrics	14
test_ollvm_protection	6
test_operation_profile	2
test_package_selection	6
test_pe_support	10
test_rpm_discovery	4
test_sample_build_privacy	9
test_security_day2	5
test_security_day3	11

Test module	Tests
test_ui_contract	8
test_ui_day2	3
test_upgrade_api	3
test_workflow	9
test_workflow_driver_scope	1
Total	167

Optional-dependency tests skip when Ghidra or document tooling is unavailable; the extracted Lite package recorded seven such skips. The suite does not execute bundled samples or third-party targets on the host; the OLLVM tests boot separate deterministic drivers in QEMU.

Appendix B. Evidence Register

The evidence register lists the artefacts a reviewer can open directly. Hashes are SHA-256; prefixes are shown here to keep the table readable, with full values in the files themselves.

Artefact	SHA-256 prefix	Proves
hardening-level5.json	cff75438814d50df8d68…	Measured ELF hardening and sample hashes
ollvm-five-qemu.json	c9e6fb1ccc3904ebb0ac…	Five deterministic driver results and scope
verification.json	8b734f4506c9d67f63da…	Recorded full-suite result
security-day4.json	3cb9b46fa993db33e9b2…	Bandit and detect-secrets summary
ACCEPTANCE_MATRIX_DAY4.md	59953fa8bc13461ecadd…	Platform and release acceptance state
CLAIM_EVIDENCE_DAY4.md	c4d269d6fc600ab485b7…	Claim-to-evidence mapping
level-1 driver (original)	f9acd983ee9555426506…	Separate deterministic driver hash
level-2 driver (original)	c31874ba506d43cb4dd9…	Separate deterministic driver hash
level-3 driver (original)	213f88ed1d92792e6494…	Separate deterministic driver hash
level-4 driver (original)	7474e58624326ead51b3…	Separate deterministic driver hash
level-5 driver (original)	d8f08e2c42759b46127e…	Separate deterministic driver hash

Bundled sample hashes

Sample	Bytes	SHA-256
level-1	13,768	55e3d47426c7385f1ad3de028e052b06…
level-2	14,432	268c6fbf9c2c775a3dd34af2066914ad…
level-3	15,168	6615b318db6254e2df469e8f2d2f6ea5…
level-4	14,784	5f02cb618c39373f8cb39246877cdc87…
level-5-hardened	12,792	2ac29fa1e499fa27bb34c6ff0d90dd2a…
level-5	15,904	7963c8cc2fdf8e09577222d8324d556e…

Appendix C. Reproduction Commands

Commands assume the repository root. They are documentation, not an invitation to execute third-party targets. The only execution permitted by the project is the QEMU sample-driver path described in Section 8.

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
python3 build_samples.py          # rebuild six self-authored samples, never run them
python3 app.py                    # http://127.0.0.1:8765/
python3 -m unittest discover -s tests -v
RUN_GHIDRA_TESTS=1 python3 -m unittest discover -s tests -v
python3 scripts/package_lite_windows.py
python3 scripts/package_evidence_day4.py
soffice --headless --convert-to pdf docs/*.docx --outdir docs
```

The test suite is safe to run on a workstation: it uses temporary directories and does not launch the bundled samples. Ghidra tests require the project-local tools and are skipped otherwise. Packaging selects Git-index files only and refuses unstaged edits to indexed source, so a dirty tree must be reviewed before packaging.

Appendix D. Security Scan Details

Bandit 1.9.4 scanned 8,859 lines of project source, tests and scripts. Total issues: 116; severity: 0 high, 3 medium, 113 low.

Rule	Count	Interpretation
B101	2	assert used in tests.
B105	2	hard-coded password string in test fixtures.
B108	1	hard-coded temporary directory in tests.
B314	2	XML parsing in tests/document tooling.
B404	26	subprocess module imported; expected for a tool that invokes static analysers.
B405	1	XML import in tests/document tooling.
B603	58	subprocess call without shell; argument-vector use is the intended pattern.
B607	24	partial executable path; resolved through a controlled tool inventory.

detect-secrets disposition

Six files produced findings. They are SHA-256 checksums in evidence/setup scripts, or deliberate fixtures used to test the sanitizer. No live credential, private key or token was found in the scanned tree. The scan excludes tools/, runs/, backups/ and .git/.

Semgrep and OWASP ZAP were used earlier in the project but were not re-run after the Day 4 changes. This report does not present them as fresh evidence.

Appendix E. Acceptance Matrix Summary

The acceptance matrix records what has been demonstrated on which platform and what remains an owner or external gate. It is the authoritative short form of this report.

Area	Fedora	Native Windows	Release gate
Workbench UI and static analysis	Verified	Not manually tested	Windows manual run
ELF analysis and hardening	Verified	PE path only on Windows	Windows PE sample

Area	Fedora	Native Windows	Release gate
PE analysis	Verified with fixtures	Not manually tested	Windows manual run
Symbol removal and hygiene	Verified	Not manually tested	Windows manual run
OLLVM source transformation	Verified on five samples	Unavailable by design	Documented limitation
QEMU driver checks	Verified, five rows	Unavailable by design	Documented limitation
Packaging and checksums	Verified locally	Extraction not tested	Windows extraction
Documents and demo	Built and reviewed	Not reviewed	Owner review
Event submission	Not started	Not applicable	Owner action

Appendix F. Glossary

Term	Meaning in this report
ELF / PE	Executable and Linkable Format (Linux) / Portable Executable (Windows).
Static analysis	Reading file bytes and metadata without running the program.
Hardening	Build/linker properties such as NX, PIE, RELRO, canary and FORTIFY.
OLLVM	Obfuscator-LLVM passes applied to source before compilation.
QEMU	Emulator used here for a no-disk, no-network guest with deterministic drivers.
Driver vector	A small program and input whose transcript is compared between builds.
Behavior verified	Reserved for the artifact actually executed; false for the downloadable copy.
N/A	Not measured in this run; never interpreted as zero.
Scope	The exact claim boundary attached to a piece of evidence.
DRAFT	Owner-review copy; not submitted, published or tagged.

This report was generated from the project's recorded evidence on 11 November 2026. It is a DRAFT for owner review.